A human-in-the-loop framework for generating and testing user-guided pedestrian scenarios from text and maps

Abstract

User studies and human-in-the-loop tests in virtual reality need *guided tasks* for one ego user in realistic traffic, such as "wait for the green light, look at the approaching car, then cross". Writing such scenarios by hand is slow, hard to reproduce, and tied to one place. We present a framework that generates them from a text prompt and an OpenStreetMap area, validates them against the map, and runs them with a human or a synthetic user as the ego pedestrian. Tasks are expressed in a small typed specification language of temporal formulas whose atoms return signed numeric margins; the language is carried inside an extension of the OpenSCENARIO and OpenDRIVE standards, and an independent standard player loads and runs the generated files. A grounded language-model agent writes the specification and a short world script in two modular stages, and a map-aware validator with a classical synthetic walker checks that the result is well formed and doable; deterministic safety nets and a standard-player gate make a valid, loadable output a property of the pipeline, with the origin of every scenario recorded. A WebXR and first-person player evaluates the same formulas live on the participant's real pose. The pipeline is modular (bare prompt, script, specification, user-defined rules, optional map image) and supports local models of at most four billion parameters as well as hosted models through an OpenAI-compatible endpoint. The same scenarios run headless for synthetic agents, which prepares learned agents as future work. We describe the method and the implementation; the experimental protocol is reported separately.

Keywords: scenario generation, human-in-the-loop, virtual reality, pedestrian, OpenSCENARIO, large language models, temporal logic

1. Introduction

Virtual reality and first-person simulation are widely used to study how people move through traffic, to train them, and to test assistive technology, for example for people with low vision. Such studies rely on *guided tasks*: the participant is told what to do ("walk to the bus stop", "wait for the green light, then cross") and the system decides whether they did it and whether they broke a rule on the way. Writing these tasks and the traffic around them by hand is slow, hard to reproduce, and tied to one place. Recent work generates driving scenarios for autonomous-vehicle (AV) testing from text with language models, but the vehicle under test is the ego and no human performs a task. Work on pedestrians in the loop, in turn, scripts its scenarios by hand.

This paper presents a framework that fills that gap. From a text prompt and an OpenStreetMap area it generates a *scenario* (road users, signals) and *guided tasks* for one ego user, validates them against the map, encodes them in an extension of the OpenSCENARIO and OpenDRIVE standards, and runs them in a WebXR and first-person player. The ego can be a human or a *synthetic user*: a classical agent that plans a route on the pedestrian graph, follows it, waits for green and yields to traffic. The synthetic user also shows that a generated scene is doable. The framework is a human-in-the-loop framework in this sense; it does not train a model, and learned synthetic users are future work that the same interface prepares.

Contributions. We claim four contributions:

1. an **extension of OpenSCENARIO and OpenDRIVE** with a typed task specification language whose formulas return signed numeric margins, evaluated by the same rules in Python and in the player (Section 4);
2. a **modular, map-grounded generation pipeline** (bare prompt, script, specification, user-defined rules, optional map image) with a validator and a repair loop (Section 5);
3. a **human-in-the-loop execution and validation framework**: a VR and first-person player that monitors the formulas live, and a classical synthetic user that checks doability (Sections 5 and 6);
4. an **evaluation** across areas, models and pipeline modules (Section 7).

A headless environment, a dataset format and reference policies are released as *enabling interfaces*; they are described briefly in Section 6 and are not claimed as contributions.

2. Related Work

2.1. Pedestrians in the loop and VR experiments

GUsT-3D [1] is the closest framework for guided user tasks in VR. It defines tasks as lists of time-quantified constraints over an annotated scene (navigation, object and environment layers; relations such as near, visible and look-at; evaluation "at a time", "at some point" or "at all times"), with a hint after a delay and failure at a time limit, and it includes an outdoor road-crossing case study. Tasks and scene annotations are authored by hand and nothing is generated. JaywalkerVR [2] puts a human pedestrian in a CARLA [3] simulation to collect safety-critical pedestrian–vehicle interactions; its scenarios are scripted and it does not define a task language. Other task-based and DSL-based embodied VR work [Robert et al.] follows the same pattern. Our task model inherits the idea of constraints with time quantifiers and the two-stage timeout from GUsT-3D, and adds a formal language with numeric margins, map-derived navigable space instead of manual annotation, generation, and a standards-based encoding.

2.2. Low-vision simulation and rehabilitation in VR

Low-vision simulators and VR rehabilitation tools supply perceptual manipulations and measure behaviour under them [low-vision simulation, rehabilitation in VR]. They need walking tasks in realistic streets but do not generate them. This line of work is a motivating application of our framework, not part of it.

2.3. Scenario specification languages and ontologies

ASAM OpenSCENARIO [4] describes the dynamic content of a scenario and ASAM OpenDRIVE [5] the road network; both target vehicle-centred simulation, and neither has a notion of a task for a human participant. Scenic [6] is a probabilistic programming language for scenes and scenarios with hard and soft constraints. Ontologies of driving scenarios [scenario ontologies] organise the concepts. Pedestrian guided tasks are not part of any of these. We keep OpenSCENARIO and OpenDRIVE as the carrier and add tasks inside a custom command (Section 4), so that tools that do not know the command can ignore it (Section 7.1 tests this).

2.4. LLM-based scenario generation

Language models now write scenario code from text. ChatScene [7] turns language into CARLA scenario programs for safety-critical testing of driving agents. ScenicNL [8] generates Scenic programs from crash reports with constrained decoding and a compiler and simulator in the loop. Text2Scenario [9] produces executable OpenSCENARIO files from text and is evaluated with a feasibility monitor, human ratings (0 to 1) of how well a recorded scenario matches its text, and construction time; Txt2Sce [10] produces OpenSCENARIO seed files from crash reports and judges the resulting runs on safety, smartness and smoothness. TARGET [11] goes from traffic rules to a validated DSL, and TTSG [12] retrieves roads and plans agents from a description. ChatDyn [13] plans multi-actor behaviour for vehicles and pedestrians from language. LLM4AV [14] is the closest pipeline-level work: text to a DSL to OpenSCENARIO on OpenDRIVE geometry with a multi-layer validator, with a pedestrian crossing among its examples. All of these generate a scenario for a vehicle under test or for scripted actors. None defines a goal-and-safety task for a human or synthetic ego pedestrian. We borrow the pattern of a validator that feeds precise errors back to the model, and the human rating of text–scenario match as one evaluation measure.

2.5. Pedestrian-focused scenario generation for autonomous-vehicle testing

Generative and adversarial models produce critical pedestrian behaviour to test AV perception and planning [Ped-Cross GAN; Suicidal Pedestrian; critical pedestrian scenarios]. There the pedestrian is the adversary of the vehicle and no human plays it.

2.6. Urban and 3D scene generation

Several systems generate urban worlds from text, maps or procedural rules for embodied agents; MetaUrban [15] is the closest in spirit, with pedestrians and micromobility agents in compositional urban scenes used for point-goal and social navigation. These systems emphasise visual and physical fidelity and agent training; they do not provide task semantics with formal pass and fail rules or a standards-based export.

2.7. Positioning and research gap

Table 1 summarises the position. Existing LLM-to-OpenSCENARIO work targets a vehicle under test; existing pedestrian VR work scripts its tasks by hand. To our knowledge, no system generates guided tasks for a

Table 1: Positioning against the closest work (what each line of work provides).

	Generated from text	Ego is a human pedestrian	Formal task pass/fail rules	Standard export
GUsT-3D [1]	–	yes	constraints	–
JaywalkerVR [2]	–	yes	–	–
ChatScene, ScenicNL [7, 8]	yes	–	–	–
Text2Scenario, Txt2Sce, LLM4AV [9, 10, 14]	yes	–	–	yes
MetaUrban [15]	–	–	–	–
This work	yes	yes	yes	yes

human or synthetic ego pedestrian from a text prompt and a real map, validates them against the map with a formal task specification, and exports them in a standards-based format that existing tools can load. We do not claim priority; the survey above is not a systematic search.

3. Problem Statement and Research Questions

Problem. Given a text prompt and an area of a real map, generate a scenario and a list of guided tasks for one ego user, such that (i) the files are valid and doable on that map, (ii) they match what the prompt asked for, (iii) a participant can play them and the system can tell, with numbers, whether each task was done and which rule was broken, and (iv) standard tools can still load the files.

Research questions.

RQ1 (standards and expressiveness). Can existing AV scenario standards (OpenSCENARIO, OpenDRIVE) be extended to express user-guided pedestrian tasks, with files that standard tooling still loads? Which task types and constraints does the extension capture?

RQ2 (generation). Can a grounded language-model agent with a constrained DSL generate valid, executable scenarios from a prompt and an OpenStreetMap area, across areas and across open and closed models?

RQ3 (modularity). What does each module contribute to validity and faithfulness to the prompt: the bare prompt template, the script pipeline, the formal specification, user-defined rules, the validator and repair loop, and a map image given to a vision-language model?

RQ4 (usability). Are the generated scenarios usable as tasks (a) for human participants (playable, completable, relevant) and (b) for synthetic agents (solvable, with scores that separate agents)?

4. Task Model and Specification Language

4.1. Guided tasks: one primitive plus constraints

Every guided task is one primitive, *reach a target*, plus constraints. Instead of fixed task classes, the constraints are written in a typed specification language, so the same machinery expresses reaching, waiting, holding, looking, following, keeping clear, pacing and signal compliance. A task has a **goal** formula (the task succeeds when it is satisfied) and a list of **safety** formulas (the task fails as soon as one is violated), two timeouts (a hint at `initial`, failure at `baseline`), a short instruction, and ordering information (`after`, sequential, parallel or independent groups). All times in formulas are seconds from the start of the task.

Constraint axes. The constraints of a task fall on six axes (Table 2). Seven named scenario types are points on these axes: point-to-point navigation over walkable ground (`ptp_env`) or onto the road (`ptp_road`), corridor following (`cpath`), obstacle avoidance (`env_con`), navigation among moving road users (`dyn_nav`), and dependent or independent composites of several tasks (`dep_comp`, `indep_comp`). The axes of any concrete scenario are *derived from its formulas*, not from per-type code, so one scenario can combine several constraints, such as crossing on green while a car approaches. Object interaction (picking up or placing), crowds, wayfinding by instruction and perception tasks are out of scope.

4.2. Atoms, combinators and definitions

Formulas are JSON objects. **Atoms** are evaluated on a *sample*: the pose, gaze and speed of the ego and the state of the world at one instant. Each atom returns a signed margin, positive when satisfied, in the unit of the quantity (metres, degrees, metres per second) or ± 1 for qualitative

Table 2: Constraint axes and when a value is derived from the formulas.

Axis	Values	Derived when
surface	walkable, crossing, road	goal zone lies on walkable ground, a crossing, or the carriageway
path	free, corridor	a path-proximity or surface atom occurs
static	none, obstacles	a distance atom refers to a non-moving entity
dynamic	none, movers	a distance atom refers to a moving entity
signal	none, obeyed	a signal atom occurs
composition	single, ordered, unordered	two or more tasks run sequentially or in parallel

atoms (Table 3). **Combinators** are $\neg$, $\wedge$, $\vee$, implication, *eventually*, *always*, *until*, *then* (stages in order) and *previous*, with time windows in seconds. A **standard library** of named definitions covers the common tasks: `reach`, `hold`, `wait_for_green`, `look`, `follow`, `keep_clear`, `pace`, `cross`, `collision_free`, `stay_on`, `stay_near_path`, `obey_signal` and `no_event`. At the highest pipeline level (L2, Section 5) the model may also write its own definitions; they stay evaluable because they may only call library definitions and earlier ones, never themselves, and only implemented atoms occur in them.

Example. "Wait for the pedestrian light, then cross, and never step onto the crossing on red" is one task with the goal `then(wait_for_green(zone, signal), cross(far_zone))` and the safety formula `obey_signal(crossing, signal)`. A second task that must follow it can ask the participant to look at an approaching car within ten seconds.

4.3. Quantitative semantics

The semantics follow the robustness of signal temporal logic [16, 17, 18], extended to *trace prefixes*, because a participant is still moving when the system must decide. Time is sampled with step $\Delta = 0.1\,\mathrm{s}$. For a formula φ

Table 3: Atoms and their margins ρ (positive means satisfied). δ is the distance from the ego to the oriented bounding box of the entity.

Atom	Fields	Margin
`in_zone`	disc, rectangle, crossing, feature, entity	$r - \|p - c\|$ for a disc; $\min(h_l - \|u\|, h_w - \|v\|)$ in the rectangle's frame
`on_surface`	surface class	$+1$ if the surface under the feet is in the class, else -1
`near_path`	path, tolerance	tolerance minus distance to the polyline
`dist_le`, `dist_ge`	entity, d	$d - \delta$, $\delta - d$
`look_at`	entity or point, angle	angle (degrees) minus the angular error between gaze and bearing
`speed_le`, `speed_ge`	v	$v -$ speed, speed $- v$
`signal_is`	signal, state	$+1$ or -1
`event`	storyboard event	$+1$ if the event has started, else -1

anchored at sample k of a prefix, the monitor returns an interval $[\underline{\rho}, \overline{\rho}](\varphi, k)$:

$$\rho(\neg\varphi, k) = -\rho(\varphi, k), \qquad \rho(\varphi \wedge \psi, k) = \min\big(\rho(\varphi, k), \rho(\psi, k)\big), \tag{1}$$

$$\rho(\Diamond_{[a,b]}\varphi, k) = \max_{j \in [k+a/\Delta,\, k+b/\Delta]} \rho(\varphi, j), \quad \rho(\Box_{[a,b]}\varphi, k) = \min_{j \in [k+a/\Delta,\, k+b/\Delta]} \rho(\varphi, j), \tag{2}$$

$$\rho(\varphi\, \mathcal{U}_{[a,b]}\, \psi, k) = \max_{j \in [k+a/\Delta,\, k+b/\Delta]} \min\Big(\rho(\psi, j),\ \min_{k \le i < j} \rho(\varphi, i)\Big). \tag{3}$$

Atoms are exact where a sample exists and $(-\infty, +\infty)$ beyond the prefix; negation swaps the bounds, conjunction and disjunction act componentwise, *eventually* takes the maximum over the known window (unknown samples can still help, so $\overline{\rho} = +\infty$) and *always* the minimum (unknown samples can still hurt, so $\underline{\rho} = -\infty$). With a complete trace $\underline{\rho} = \overline{\rho}$ and this is the usual robustness. The decision rule for a task is: it *fails* as soon as a safety formula has $\overline{\rho} < 0$ (the violated rule's label, for example "crossed on red", is reported); it *succeeds* when the goal has $\underline{\rho} \ge 0$; it fails when the goal has $\overline{\rho} < 0$ or at the baseline timeout; otherwise it is undecided. Success and failure are therefore detected at the earliest moment they are determined, not only at the end, and the participant gets numbers (metres, degrees, metres per second) instead of a binary outcome.

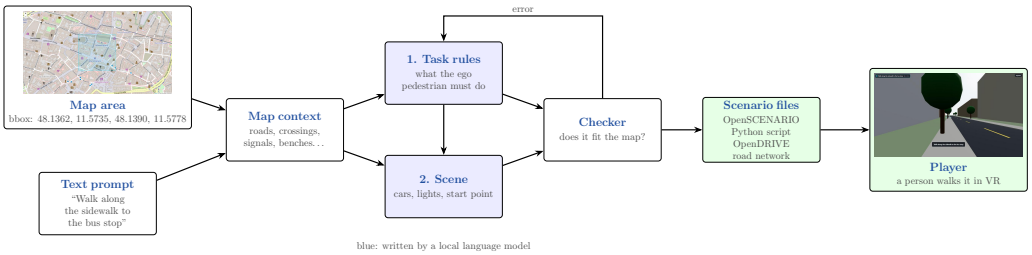


Figure 1: The pipeline. A map area and a text prompt give a map context; a language model writes the task rules (stage 1) and the scene (stage 2); a checker validates both against the map and returns errors for repair; the result is a set of standard files that a player runs with a person in VR or first person. Blue boxes are written by the language model.

4.4. Encoding in OpenSCENARIO and OpenDRIVE

A scenario is written as OpenSCENARIO 1.3: inline pedestrian and vehicle entities, initial teleports and signal states, stories for the user tasks, for scripted trajectories of other road users and for signal phases, and a stop trigger. The tasks of the ego pedestrian are `UserDefinedAction/CustomCommandAction` elements of type `VR_TASK` with a real `VRTask` child (goal, target, instructions, feedback, dependencies, group mode, timeouts) and a `Spec` element that holds the goal and safety formulas as a tree of `F` elements. Task order uses the standard `StoryboardElementStateCondition`. Because the standard defines custom commands as an extension point, tools that do not know them ignore the element; Section 7.1 checks this with an independent player. Traffic signals follow what standard consumers expect: numeric OpenDRIVE signal identifiers, which `TrafficSignalStateAction` also uses as its name, and lamp-state strings of the signal type ("on;off" and "off;on" for red and green on a two-lamp pedestrian light). The symbolic names that formulas and the player use are declared in the file as parameters that map each name to its numeric identifier, so that the player can map them back and files written earlier with symbolic names still parse. The map is exported as OpenDRIVE with the same metric coordinates: roads trimmed at junctions, junction connecting roads, crossings as objects and traffic lights as signals.

5. Framework

5.1. One frame, one map

OpenStreetMap [19] data is projected once into a local East–North frame in metres (heading 0 is east, counter-clockwise). Everything reads that metric model: the validator, the agent's tools, the synthetic user, the player, the scene map exchanged with the player and the OpenDRIVE export. The map is rounded the way it is serialised before validation, so the sandbox, which loads the serialised file, and the host see identical numbers.

5.2. Modular generation pipeline

The model works as an agent that receives a pre-fetched map overview, an inventory of what the map contains, and a working reference for the most likely scenario type; it adapts the reference, calls grounded tools when it needs an identifier (`list_features`, `check_point`, `plan_route`, `list_atoms`), and repairs from the validator's messages. The pipeline has modular levels, which are also the conditions of the ablation (RQ3):

T0 (bare template). One model call with the same context as L0, no tools and no repair; the reply is validated once. This is what a user gets by pasting the template into a chat model.

L0 (script). The model fills in a Python script that calls task helpers (`go_to`, `cross`, `follow_path`) of the scenario library. This is the original system; the helpers compile to the same formulas as the higher levels.

L1 (specification). Stage A: the model writes a *specification* as data from the standard library; it is never executed. Stage B: it writes a short world script (user, vehicles, signals) that calls `scenario.apply_spec(SPEC)`.

L2 (own definitions). As L1, and the model may define its own library entries.

Vision. Each level can additionally receive an annotated top-down map image (north up, metric grid, identifiers of crossings and stops) when the model accepts images.

The model supplies content only; the scenario builder assembles the storyboard deterministically. If a script omits the user while applying a specification, the participant is placed about 18 m of route distance before the first goal zone.

5.3. Validation and the repair loop

Stage A is parsed, type-checked and resolved against the map without being executed: zones refer to real identifiers, walking feasibility is checked, and the check tells the model which entities and signals the world script must create. Stage B runs in a sandbox. A map-aware validator then checks geometry, surface rules and task feasibility; the checks that used to be seven hand-written rules, one per scenario type, are now *derived from the formulas* (for example a keep-clear rule on a moving entity needs a mover whose route meets the participant's path). Every error is returned to the model as a precise message, and the model repairs its submission.

5.4. Safety nets and the standard-player gate

A pipeline that relies on a small model cannot guarantee a result, because a model can fail at the plumbing (a wrong call to the map interface, a missing entity) or stop answering. Two deterministic nets sit around the model, in order, and every result records which one produced it (its *tier*):

model the model's own submission was accepted; no net was involved.

assisted the model's specification was accepted but its world script kept failing, so the harness writes the world script from what the specification needs (the signals, and the vehicles, walkers or parked cars it names, placed around the crossing its zones refer to). The task rules are still the model's.

fallback the model produced nothing usable, so the harness submits the reference script of the most likely scenario type. It is valid on every map that has the features the type needs but need not match the request.

The nets can be switched off, to measure the model alone, or on, to guarantee a valid output. Because a fallback scenario is valid without being faithful, evaluation reports two quantities: the acceptance of the model on its own, and the valid output of the pipeline with its tier. Independently of the nets, a scenario can be required to pass a *standard-player gate*: it is accepted only if an independent player (esmini) loads it and runs it to its end, and a failure is returned to the model like any other validator error. The map interface is also tolerant: it accepts the records returned by its own queries, coordinate pairs and identifiers wherever a place or an identifier is expected, which removes a frequent class of script errors.

5.5. The synthetic user

A classical *nominal walker* acts as the ego. It plans a route on the pedestrian graph, follows it through the goal zones in formula order, waits where the formula asks (hold windows, "until the signal is green", gated crossings), yields to approaching traffic and keeps its gaze on look targets. The same monitor evaluates the formulas on its trace; a negative margin is reported to the model as a warning that the specification may not be doable. No learning is involved. In the validator it is an offline device that never moves the human participant. The same walker, closed-loop, is the scripted reference policy of the headless interface (Section 6).

5.6. Live monitoring of the human ego

In the player the participant is the ego. The player evaluates every task's formulas at 10 Hz against the *real* pose (head pose in VR, camera in first person), displays the instruction and hints, and reports success, the label of the violated rule or a timeout, together with the margins. Results (per-task outcome and margins) and a 10 Hz trace can be downloaded.

6. Implementation

The software consists of a Python package (`usg`) and a TypeScript player. This section summarises the components that matter for reproducing the method.

6.1. Map ingestion and the metric map model

OpenStreetMap extracts are parsed into a map model of roads (split at shared nodes and trimmed at junctions), sidewalks (offset polylines; a default of 2 m on residential and secondary roads when untagged), crossings (a rectangle with centre, heading, length and width), traffic signals, props (benches, bus stops, lamps, trees, bollards, waste baskets), buildings, areas and paths. A single function classifies any ground point as crosswalk, building, water, road, sidewalk, path or ground with a fixed priority; it is the source of truth for the validator, the walker and the player. A pedestrian routing graph with a crossing penalty, searched with A*, supports feasibility checks and the nominal walker. The map can be selected in the interface with a search bar and a rectangle (at most 1 km^2) or loaded from a file.

6.2. Scenario DSL and storyboard assembly

The scenario library extends the `scenariogeneration` package with a pedestrian-oriented DSL: `MAP` queries (crossings, roads, kerbs, routes, walking times), a scenario builder with `user`, `vehicle`, `parked_vehicle`, `walker`, `signal_cycle` and the task helpers, and `apply_spec` for specification documents. The builder produces the OpenSCENARIO file deterministically, together with the OpenDRIVE file and a JSON scene map for the player. The seven reference scripts (one per scenario type) adapt to any map by selecting features from it and serve as few-shot examples.

6.3. Sandboxed execution

Generated scripts are plain Python. They pass an AST policy (allowed imports, no file access, no classes, no private names) and run in a subprocess with a restricted runtime and process and memory limits. Specifications are data and are never executed.

6.4. Language models and the agent loop

The agent speaks to any OpenAI-compatible chat endpoint. Local models run through LM Studio's llama.cpp engine, started from the user's model files; hosted open or closed models can be used through the same client by changing the endpoint, with a plain request body for hosted services. Tool calls use a plain-text format (a `tool` tag with a JSON argument) rather than native function calling, because small models handle it more reliably. The harness performs the two obvious first calls (map overview, reference script or specification) itself, so the model does not spend steps exploring. The conversation is compacted to fit the model's context.

6.5. Interactive player

The player is a React, three.js and WebXR (react-three-xr) application served by the same server as the Gradio interface. It parses the scenario including the `Spec` formulas, interprets acts, events and conditions, moves vehicles and walkers, cycles signals and builds procedural 3D meshes from the map (props are placeholders that can be replaced by glTF assets). Its monitor is a TypeScript twin of the Python monitor. Parity is tested: the surface classifier agrees with the Python one on 13,440 sampled points, and the monitors agree on 160 random formulas under 12 prefix and anchor settings. An inspector shows the live pose, every task's formulas with their current margins, and all storyboard events.

6.6. Headless interface for synthetic agents

The same formulas drive a headless, Gymnasium-style environment: a scenario specification and its map give `reset` and `step` with an action of speed, turn and gaze; the reward is the improvement of the best goal margin so far plus a bonus per completed task and -1 on a rule violation or timeout, and a separate cost marks safety violations; observations are fixed-size arrays with padded and masked entity and signal lists. Vehicles and walkers follow their scripted routes and do not react to the ego. A dataset builder writes scenarios, maps and an index with splits by map, and reference policies (random, straight-line, scripted route-following) are provided. This interface prepares learned agents as future work (Section 8.2).

6.7. Testing

The Python package has automated tests for the map, the OpenDRIVE export, the seven reference scenarios, the validator, the sandbox and its security policy, the specification language, the agent loop at all levels, the language-model client, the headless environment and the dataset builder; the player has TypeScript tests for the parity checks above, scenario parsing and task rules.

7. Evaluation

[TODO: Experimental protocol: areas, prompt sets, models (open local up to 4B, larger open, closed), modular conditions T0/L0/L1/L2 with and without the map image, metrics (validity, first-try validity, adherence, axes recall, nominal-walker margin, human rating of text–scenario match), statistics. To be planned together.]

7.1. Compatibility with a standard player (RQ1)

We checked generated files with esmini [20] v3.10.2, an independent OpenSCENARIO player and OpenDRIVE road manager, on 69 scenarios from two areas (Sophia Antipolis and a synthetic city): the seven reference scenarios, and three specification-based ones with traffic lights, a gaze rule and a moving car, with four variants each and duplicates removed. A scenario passes if esmini loads it as OpenSCENARIO 1.3, runs it to its end with exit code 0 and logs no error.

OpenSCENARIO.. All 69 scenarios pass in the final run, including the 16 with traffic-light actions. The first run passed only 53: all 16 scenarios with traffic-signal actions were rejected. We isolated two causes on one scenario. First, esmini converts the name of a `TrafficSignalStateAction` to an integer, but we wrote symbolic signal identifiers. Second, once identifiers were numeric, esmini expected the state to be a lamp mode of its own traffic-light types ("on", "off", ...) and not "red" or "green"; it also rejected a `value` attribute on the OpenDRIVE signal. The exporter now writes numeric identifiers and lamp states, declares the symbolic names as parameters, and the player maps them back (Section 4.4). The `VR_TASK` custom commands with their `VRTask` and `Spec` children are ignored by esmini without failing, as the extension intends. The remaining warnings are harmless: esmini cannot locate 3D models, and four vehicle routes report "no connection" between lanes of different roads.

OpenDRIVE.. esmini's road manager loads both maps and finds as many roads as the file contains (824 of 824 for Sophia Antipolis, 24 of 24 for the synthetic city). To test that the export and the metric map describe the same geometry, we projected points on our road centre lines onto esmini's road geometry: the mean distance to the reference line esmini assigns is 0.08 m over 1344 samples for Sophia Antipolis, 1344 minus 25 samples (98.1%) lie within 0.5 m, and the largest distance is 6.3 m; the cause of the outliers was not investigated (candidates are roads that overlap or run close together at junctions). All 36 samples of the synthetic city match exactly. The check also exposed a defect of our exporter that we fixed before the final run: a crossing centred in a short junction-trimmed road was placed beyond the end of that road.

Not covered.. The files were not validated against the ASAM schemas (they are licensed separately), only one version of one player was tried, and the viewer of esmini crashes in the headless container used here, also on a minimal valid OpenDRIVE file.

7.2. Valid output by construction (RQ2, RQ3)

We replayed the 36 runs of the first experiment (level L2, Sophia Antipolis, one 4B model, 75 runs) that the model did not finish through the safety nets of Section 5.4, offline and with the standard-player gate switched on. The runs had failed because the model kept writing scripts that the validator rejected (19), stopped calling tools (9), triggered a server error on an

over-long request (4), reached the step limit (2) or kept submitting rejected specifications (2). All 36 ended in a valid scenario that esmini also loads and runs: 2 were assisted (the harness wrote the world script around the model's specification; 1 of them matches the request) and 34 were fallbacks (12 match the request). Together with the model's own 39 acceptances this gives 75 valid outputs in 75 runs, of which 39 are successes of the model. The replay measures the nets, not the model: a fallback is valid but mostly does not match the request, and the 39 of 75 model acceptances (52%) is the figure that describes the model. The two quantities are therefore always reported separately, with the tier of every scenario.

7.3. Benchmark: validity and prompt adherence (RQ2, RQ3)

[TODO: Preliminary result from the first experiment (one area, one 4B model, level L2) shown in Table 4; the multi-area, multi-model and ablation results are to be run.]

Table 4: Performance of the L2 pipeline per scenario type on the Sophia Antipolis map. Result: count (percentage; 95% Wilson interval) over all runs of that type.

Scenario type	Task description	Quantitative metric	Result
`ptp_env` (9 runs)	Reach a landmark (bus stop, bench, waste basket) over walkable ground	Accepted (validator passes the final scenario)	9/9 (100%; 70–100)
		First try (accepted with no repair loop)	3/9 (33%; 12–65)
		Adherent (constraint axes and operators match the request)	9/9 (100%; 70–100)
		Solvable (the nominal walker satisfies all tasks)	9/9 (100%; 70–100)
		Script submissions per run (mean)	1.33

Scenario type	Task description	Quantitative metric	Result
		Model steps per run (mean)	3.7
		Wall-clock per run, s (mean)	44
`ptp_road` (9 runs)	Reach a spot on the carriageway	Accepted (validator passes the final scenario)	6/9 (67%; 35–88)
		First try (accepted with no repair loop)	4/9 (44%; 19–73)
		Adherent (constraint axes and operators match the request)	5/9 (56%; 27–81)
		Solvable (the nominal walker satisfies all tasks)	5/9 (56%; 27–81)
		Script submissions per run (mean)	0.89
		Model steps per run (mean)	6.4
		Wall-clock per run, s (mean)	52
`cpath` (9 runs)	Reach a goal while staying inside a corridor (path or sidewalk)	Accepted (validator passes the final scenario)	1/9 (11%; 2–44)
		First try (accepted with no repair loop)	0/9 (0%; 0–30)
		Adherent (constraint axes and operators match the request)	1/9 (11%; 2–44)
		Solvable (the nominal walker satisfies all tasks)	1/9 (11%; 2–44)
		Script submissions per run (mean)	4.89

Scenario type	Task description	Quantitative metric	Result
		Model steps per run (mean)	6.1
		Wall-clock per run, s (mean)	121
`env_con` (9 runs)	Reach a goal without touching static obstacles (parked cars)	Accepted (validator passes the final scenario)	4/9 (44%; 19–73)
		First try (accepted with no repair loop)	2/9 (22%; 6–55)
		Adherent (constraint axes and operators match the request)	4/9 (44%; 19–73)
		Solvable (the nominal walker satisfies all tasks)	4/9 (44%; 19–73)
		Script submissions per run (mean)	3.11
		Model steps per run (mean)	6.8
		Wall-clock per run, s (mean)	185
`dyn_nav` (9 runs)	Cross while keeping clear of a moving vehicle	Accepted (validator passes the final scenario)	5/9 (56%; 27–81)
		First try (accepted with no repair loop)	2/9 (22%; 6–55)
		Adherent (constraint axes and operators match the request)	5/9 (56%; 27–81)
		Solvable (the nominal walker satisfies all tasks)	4/9 (44%; 19–73)

Scenario type	Task description	Quantitative metric	Result
		Script submissions per run (mean)	4.33
		Model steps per run (mean)	5.6
		Wall-clock per run, s (mean)	185
`dep_comp` (9 runs)	Several goals that must be reached in a given order	Accepted (validator passes the final scenario)	5/9 (56%; 27–81)
		First try (accepted with no repair loop)	0/9 (0%; 0–30)
		Adherent (constraint axes and operators match the request)	5/9 (56%; 27–81)
		Solvable (the nominal walker satisfies all tasks)	5/9 (56%; 27–81)
		Script submissions per run (mean)	4.44
		Model steps per run (mean)	8.2
		Wall-clock per run, s (mean)	192
`indep_comp` (9 runs)	Several goals that may be reached in any order	Accepted (validator passes the final scenario)	7/9 (78%; 45–94)
		First try (accepted with no repair loop)	0/9 (0%; 0–30)
		Adherent (constraint axes and operators match the request)	0/9 (0%; 0–30)
		Solvable (the nominal walker satisfies all tasks)	7/9 (78%; 45–94)

Scenario type	Task description	Quantitative metric	Result
		Script submissions per run (mean)	2.22
		Model steps per run (mean)	10.2
		Wall-clock per run, s (mean)	139
`mixed` (12 runs)	Combined constraints: signal compliance, a moving car, looking at it	Accepted (validator passes the final scenario)	2/12 (17%; 5–45)
		First try (accepted with no repair loop)	0/12 (0%; 0–24)
		Adherent (constraint axes and operators match the request)	2/12 (17%; 5–45)
		Solvable (the nominal walker satisfies all tasks)	0/12 (0%; 0–24)
		Script submissions per run (mean)	2.58
		Model steps per run (mean)	6.9
		Wall-clock per run, s (mean)	224

7.4. Synthetic users and human participants (RQ4)

[TODO: Synthetic agents in the headless environment; pilot with participants.]

8. Discussion

[TODO: Discussion of results.]

8.1. Limitations

Virtual-reality sessions on hardware are untested: the player runs and renders in a desktop browser, but entering a headset session, controller locomotion and the in-VR text display have not been tried on a headset. Props are procedural placeholders. The nominal walker is an idealised participant; a warning from it means "check the specification", not "impossible". Gaze is the camera yaw (head orientation in VR, mouse on the desktop); eye tracking is not used. Map quality depends on OpenStreetMap tags. The request checks that decide whether a scenario matches a prompt are keyword based. Other road users follow scripted routes and do not react to the ego. The OpenDRIVE export is checked structurally and with an independent road manager and player (one version of one tool), not against the schema. Small models follow nuanced prompts only partly; the validator is the quality gate.

8.2. Applications and future work

The framework is meant for VR user studies, including low-vision and rehabilitation research, for human pedestrians in interaction tests with simulated vehicles, and as a source of rule-constrained navigation tasks for synthetic agents such as robots, delivery vehicles or wheelchairs, where a formal specification gives measurable compliance. Future work includes learned or imitation-learned synthetic users and learned generators, reactive traffic, eye tracking, and object-interaction tasks.

9. Conclusion

[TODO: Conclusion after the evaluation.]

CRediT authorship contribution statement

Declaration of competing interest

Funding

Data availability

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

Appendix A. Evaluation prompts

Table A.5: Evaluation prompts (Sophia Antipolis map, all requested object types exist on it).

Id	Type	Prompt
sa_ptp_env_1	ptp_env	Walk along the sidewalk to the nearest bus stop.
sa_ptp_env_2	ptp_env	The participant should find the bench nearby and walk up to it.
sa_ptp_env_3	ptp_env	Go to the waste basket and stop next to it.
sa_ptp_road_1	ptp_road	Walk out onto the road and stand on the marked spot just past a crossing.
sa_ptp_road_2	ptp_road	The participant must stand in the middle of the carriageway for a moment.
sa_ptp_road_3	ptp_road	Have the user walk onto the road surface at the far end of the street.
sa_cpath_1	cpath	Cross the road but stay inside the marked corridor along the crosswalk.
sa_cpath_2	cpath	Follow the marked path along the street without leaving it.
sa_cpath_3	cpath	Walk to the crossing using only the sidewalk; stepping off it means failure.
sa_env_con_1	env_con	Cross the road at the zebra crossing without touching the parked cars.
sa_env_con_2	env_con	Walk between the parked cars along the kerb and reach the other end without touching any of them.
sa_env_con_3	env_con	Avoid the parked vehicles along the street while you walk to the bus stop.
sa_dyn_nav_1	dyn_nav	Cross at the zebra crossing while a car approaches; avoid being hit.

Table A.5: Evaluation prompts (Sophia Antipolis map, all requested object types exist on it).

Id	Type	Prompt
sa_dyn_nav_2	dyn_nav	A bus drives along the street. Walk across the road without being hit by it.
sa_dyn_nav_3	dyn_nav	Cross the road with oncoming traffic and keep clear of the speeding car.
sa_dep_comp_1	dep_comp	First walk to the crossing, then cross the road, then continue to the spot on the other side.
sa_dep_comp_2	dep_comp	Do three things in order: go to the bus stop, then to the bench, then to the waste basket.
sa_dep_comp_3	dep_comp	Walk to the kerb, afterwards cross the road, and finally walk to the waste basket.
sa_indep_comp_1	indep_comp	Visit the waste basket and the bench, in any order.
sa_indep_comp_2	indep_comp	Find both the bench and the bus stop; the order does not matter.
sa_indep_comp_3	indep_comp	Reach a bench and a tree, either one first.
sa_mixed_1	mixed	Wait for the pedestrian light to turn green, then cross the road.
sa_mixed_2	mixed	Cross at the signalised crossing on green while a car comes down the street, and stay clear of the car.
sa_mixed_3	mixed	Look at the approaching car, then cross the road once it has passed.
sa_mixed_4	mixed	At the crossing wait until the light is green, look at the traffic light, and then cross.

References

[1] H.-Y. Wu, F. Robert, T. Fafet, B. Graulier, B. Passin-Cauneau, L. Sassatelli, M. Winckler, Designing guided user tasks in VR embodied experiences, Proceedings of the ACM on Human-Computer Interaction 6 (EICS) (2022) 1–24. doi:10.1145/3532208.

[2] K. Mukoya, E. Weng, R. Choudhury, K. Kitani, JaywalkerVR: A VR system for collecting safety-critical pedestrian-vehicle interactions, preprint (2024). arXiv:2407.04843.

[3] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, V. Koltun, CARLA: An open urban driving simulator, in: Proceedings of the 1st Conference on Robot Learning (CoRL), Vol. 78 of PMLR, 2017, pp. 1–16.

[4] ASAM e.V., ASAM OpenSCENARIO, standard for the description of dynamic content of driving and traffic simulation scenarios; this work targets version 1.3.

[5] ASAM e.V., ASAM OpenDRIVE, standard for the description of road networks; this work exports revision 1.6.

[6] D. J. Fremont, T. Dreossi, S. Ghosh, X. Yue, A. L. Sangiovanni-Vincentelli, S. A. Seshia, Scenic: A language for scenario specification and scene generation, in: Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI), 2019. doi:10.1145/3314221.3314633.

[7] Zhang, Xu, Li, ChatScene: Knowledge-enabled safety-critical scenario generation for autonomous vehicles, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, arXiv:2405.14062.

[8] K. Elmaaroufi, et al., ScenicNL: Generating probabilistic scenario programs from natural language, in: Conference on Language Modeling (COLM), 2024, arXiv:2405.03709.

[9] Text2Scenario: Text-driven scenario generation for autonomous driving test, preprint; author list to be completed from Zotero (2025). arXiv:2503.02911.

[10] Txt2Sce: Scenario generation for autonomous driving system testing based on textual reports, preprint; author list to be completed from Zotero (2025). arXiv:2509.02150.

[11] TARGET: Traffic rule-based test generation for autonomous driving via validated LLM-guided knowledge extraction, preprint; author list and year to be completed from Zotero. arXiv:2305.06018.

[12] B.-K. Ruan, H.-T. Tsui, Y.-H. Li, H.-H. Shuai, Traffic scene generation from natural language description for autonomous vehicles with large language model, preprint (2024). arXiv:2409.09575.

[13] ChatDyn: Language-driven multi-actor dynamics generation in street scenes, preprint; author list to be completed from Zotero (2024). arXiv:2412.08685.

[14] LLM4AV: text to a domain-specific language to OpenSCENARIO on OpenDRIVE geometry with a multi-layer validator, springer book chapter, doi 10.1007/978-981-95-6426-2_9; cited from its abstract, to be read in full; title and authors to be completed from Zotero (2026).

[15] W. Wu, H. He, J. He, Y. Wang, C. Duan, Z. Liu, Q. Li, B. Zhou, MetaUrban: An embodied AI simulation platform for urban micromobility, in: International Conference on Learning Representations (ICLR), 2025, arXiv:2407.08725.

[16] O. Maler, D. Nickovic, Monitoring temporal properties of continuous signals, in: Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems (FORMATS/FTRTFT), Vol. 3253 of LNCS, 2004, pp. 152–166.

[17] A. Donzé, O. Maler, Robust satisfaction of temporal logic over real-valued signals, in: Formal Modeling and Analysis of Timed Systems (FORMATS), Vol. 6246 of LNCS, 2010, pp. 92–106.

[18] G. E. Fainekos, G. J. Pappas, Robustness of temporal logic specifications for continuous-time signals, Theoretical Computer Science 410 (42) (2009) 4262–4291.

[19] OpenStreetMap contributors, OpenStreetMap, https://www.openstreetmap.org, data available under the Open Database License.

[20] esmini: a basic OpenSCENARIO player, https://github.com/esmini/esmini, release v3.10.2 used for the compatibility check.